

Physical layer. This layer deals with physical characteristics of the transmission medium. It deals with the actual cabling, network cards, hubs and other devices that provide physical connection between the client and the Web server.

Fig. 12 shows how data is transmitted between networked devices.

Software program

The coding is done in 'C' language and compiled using Turbo C. Turbo C has a good collection of graphics libraries, which have been utilised for graphics programming by using basics of 'C'. The program uses various 'C' graphic files such as *egavga.bgi*, *scri.chr*, *trip.chr* and *tsr.chr*.

When the program is run, the program output will be as shown in Fig. 13. The first program output shows peer-to-peer network, where multiple peers are connected to one another. All the nodes are referred to as peers as they are equally responsible in sharing information.

The next program output shows a server along with multiple clients that can be connected over a network in order to share information.

Next, the program shows client-server model that uses a client, a server and a data source. The client sends a query to the server over a network (refer Fig. 14). The server then looks up the data in order to respond to the client's query (Fig. 15).

The next screen (Fig. 16) shows the seven layers in the OSI model.

The final screen shows how a message travels over the different layers of a network. The sender sends a message that flows from its application layer to physical layer on the sender side. Further, through physical layer, it establishes connection with the receiver to transmit the message over the layers of the receiver and eventually to the receiver.

Main functions used

Initgraph (&Gd, &gm, "c:\turbo3\bg"). Initialises the graphics system by loading a graphics driver from the disk

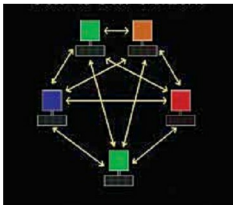


Fig. 13: Program output for peer-to-peer network

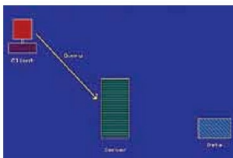


Fig. 14: Client query model

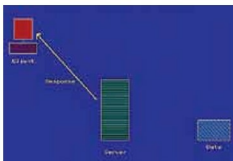


Fig. 15: Server response model



Fig. 16: Program output for OSI model

and then putting the system into graphics mode. It also resets all the graphics settings like colour, palette and current position to their default values.

***gd**. Specifies the graphics driver to be used.

***gm**. Specifies the initial graphics mode (unless *gd = DETECT). When

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download on www.efymag.com website.

***gd** = DETECT, **initgraph** sets graphics mode to the highest resolution available for the detected driver.

Void projectwork1(). Demonstrates peer-to-peer network.

Void projectwork2(). Displays server and multiple clients.

Void projectwork3(). Demonstrates how a client receives response of the query made to the server.

Void projectwork4(). Displays the seven layers of OSI model.

Void projectwork5(). Demonstrates transmission flow of the message from the sender to the receiver.

Closegraph(). Switches the screen from graphics mode back to text mode. It is called after **getch()** so that the screen does not clear until the user hits the key.

Setbkcolor(number). Sets the background colour of choice from colour options 0 through 15. Instead of specifying the number, the respective colour name can also be used in block letters from the specified set.

SetColor(number). Selects a foreground or drawing colour from the same set of 16 colours.

SetTextstyle(font,direction,character size). Sets the values for text characteristics to be used for displaying the text.

Outtextxy(x,y,"Literal expression"). Displays the string in graphics mode. (x,y) denotes the starting coordinate of the string on the screen.

Setfillstyle(pattern,color). Used to fill colours. It also allows to apply fill patterns of colouring.

Floodfill(). Used to fill the required colour in geometrical figures. Geometrical figures can be drawn using the inbuilt functions of shapes (rectangle, circle, etc). ●

The author is a B.Tech in electronics & communication. He likes writing articles and computer programming